SOP-SA-05 — Structured Logging & Error Handling

Kode	SOP-SA-05
Pemilik	Saitama (Senior Backend Engineer)
Revisi	1.0 · **Berlaku** 2026-05-29 · **Reviu** per kuartal
Referensi	12-Factor App (log as stream), ISO/IEC 27001 (logging & monitoring), tools/logging-standard.md, tools/error-code-table.md
COBIT	DSS05 (Managed Security Services)

1. Tujuan

Menjamin tiap incident bisa **ditelusur dari log** (req-id + user-id + action + outcome) dan tiap error punya **kode stabil** yang jadi kontrak debugging FE/BE — supaya gak ada `catch` diam, gak ada `console.log` di prod, gak ada incident yang gelap.

2. Ruang Lingkup

- **Berlaku:** semua endpoint, background job, auto-action, error path BE.
- **Tidak berlaku:** prototype Fauxbase-only di FE (logging nyusul pas real BE).

3. Definisi & Istilah

- **Structured logging** — log format JSON dengan field konsisten (req-id, user-id, action), bukan teks bebas.
- **Error code** — kode stabil (cth `VALIDATION_ERROR`, `CONFLICT`) yang gak berubah, jadi kontrak FE.
- **RED metric** — Rate, Errors, Duration (3 metrik kesehatan service).
- **Catch diam** — `catch (e) {}` yang nelan error tanpa log/rethrow = anti-pattern.

4. Referensi

12-Factor App (faktor XI: log as event stream ke stdout), ISO/IEC 27001 (logging & monitoring failures), OWASP (error message gak leak stack/secret), Tata mandate (evidence-first, no silent auto).

5. Tanggung Jawab (RACI)

Aktivitas	R	A	C	I
Standar log + error code	Saitama	Saitama	@levi (observability stack)	@killua (consume error code)
Implementasi log/error per endpoint	Saitama	Saitama	—	@l
Trace incident dari log	Saitama	@kakashi (RCA)	@levi	Tata

6. Prosedur

6.1 Logging

- 6.1.1 Pakai **structured logger** (Pino/winston), output JSON ke stdout (12-Factor). **Decision point:** ada `console.log`? → ganti logger + log level (kontrol: no console.log di prod).
- 6.1.2 Tiap request log wajib field: `request_id`, `user_id`, `action`, `outcome`, `latency_ms` (logging-standard).
- 6.1.3 **Auto-action di-log eksplisit** (koord SOP-SA-04 §6.4) — siapa-trigger, kapan, kenapa.
- 6.1.4 **Jangan log secret/PII mentah** (password, token, nomor kartu) — redact (OWASP/27001).

6.2 Error handling

- 6.2.1 **Catch spesifik**, bukan catch-all diam. Unknown error → log + rethrow (jangan ditelan). **Exit kalau nemu `catch {}` kosong → fix.**
- 6.2.2 Map error → **status code + error code stabil** (error-code-table): `400` syntactic, `422` semantic, `401/403` auth, `409` conflict, `429` rate-limit, `500` unexpected, `503` degraded.
- 6.2.3 **Error message ke client gak leak** stack trace / detail internal / secret (OWASP Security Misconfiguration).
- 6.2.4 Transient error (network, lock) → retry dengan backoff; permanent error → fail fast + log.

6.3 Observability

- 6.3.1 Expose **health check** `/health` (state komponen) + **RED metric** (rate/errors/duration) — koord @levi.
- 6.3.2 **Exit criteria:** ambil 1 endpoint, trigger error → cek log JSON lengkap (req-id+user-id+action) + error code konsisten dengan tabel.

7. Pengecualian

- **Debug lokal:** boleh verbose log, tapi **gak boleh nyebrang ke prod** (level guard).

8. Rekaman & Retensi

Rekaman	Lokasi	Retensi
Standar logging + error code table	`tools/` / doc	Living
Log produksi	observability stack (Levi)	Sesuai retensi infra
Trace incident (evidence)	`log.md` + RCA (Kakashi)	Permanen

9. Indikator Kinerja (KPI)

KPI	Rumus	Target
Trace-ability incident	# incident BE yang akar ketemu dari log ÷ total	↑ → 100%
Catch diam	# `catch {}` kosong di codebase	**0**
Console.log di prod	# `console.log` di kode prod	**0**
Error code consistency	# error response pakai kode dari tabel ÷ total error	100%

10. Riwayat Revisi

Rev	Tanggal	Perubahan
1.0	2026-05-29	Terbit pertama